

ISE MASTER CHEAT SHEET (INHNO006, Ctrl+F friendly)

Built from: lectures L02 to L11, tutorial T01, and the 3 real exam sittings (SS25 Final, Oct 2025 Retake, SS26 Exam). How to use: press Ctrl+F and type a TAG (like #easymock) or a KEYWORD from the lookup table below. Marks: (EXAM) = taken from a real graded exam. (LECTURE) = from the slides. (GENERAL) = standard knowledge, double check with slides.

#index TAG INDEX

#rules top rules | #history what each exam contained | #lookup keyword to tag table #junit #easymock #whitebox #bva #ep (testing code) | #rest #spring #webflux (REST) #acceptance #gherkin | #solid #design #patterns #observer | #architecture #ddd #requirements #quality #qa #ci #deploy #git #process #scrum #comprehension #uml #deployment #communication #activity #class #sequence #usecase #modelprops #quizbank (every past question with answer) | #confusion (swapped concepts) | #heuristics | #mistakes

#lookup KEYWORD TO TAG (if the question says X, search Y)

Question mentions	Search
mock, EasyMock, @Mock, expect, replay, verify, not called, capture	#easymock
assertEquals, assertThrows, @BeforeEach, delta	#junit
coverage, line coverage, statement, branch, how many %	#whitebox
equivalence class, boundary, valid/invalid, three-point	#bva
@RestController, status code, 404, 400, 204, @RequestParam, ResponseStatusException	#rest
WebClient, Consumer, async, non-blocking, Mono, Flux	#webflux
Gherkin, Given When Then, Cucumber, BDD, Three Amigos, FitNesse	#gherkin
Observer, notify, update, views, subject	#observer
SOLID, OCP, DIP, LSP, ISP, SRP, switch on type, new inside class	#solid
rename, readable, refactor, Extract Method, naming, comment	#design
singleton, facade, state, strategy, factory, adapter	#patterns
layers, microservices, pipes and filters, Conway, ADR	#architecture
aggregate, bounded context, core domain, value object, event storming	#ddd
INVEST, user story, functional, non-functional, persona, SRS, use case	#requirements
ISO 25010, usability, reliability, security, performance, safety, fault, failure	#quality
review, inspection, Fagan, static analysis, validation, verification	#qa
CI, pipeline, Gradle, build script, GitLab	#ci
blue-green, canary, dark launching, A/B, feature toggle, continuous delivery	#deploy
git add, commit, push, pull, fetch, branch, merge, rebase, non-fast-forward	#git
waterfall, Scrum, Kanban, sprint, iterative, incremental, prototype, spiral, Delphi, planning poker	#process
cyclomatic, cognitive, McCabe, naming, comprehension	#comprehension

Question mentions	Search
deployment diagram, device, artifact, node, lollipop	#deployment
communication diagram, numbered messages	#communication
activity diagram, fork, merge, decision	#activity
model, original, abstraction, pragmatics, descriptive, prescriptive	#modelprops
maintenance (corrective, adaptive, perfective, preventive)	#process (maintenance)

#rules TOP RULES (read first)

1. COMMIT AND PUSH every code exercise. No commit = 0 points. REST scored 0 in all three sittings because nothing was committed. Check on Artemis that the commit shows up. IntelliJ: Ctrl+K (commit), Ctrl+Shift+K (push).
2. Grading is per test or per method. One broken line that breaks 4 tests = 4 tests lost. No partial credit.
3. Do exactly what the task text says. Copy formulas and numbers literally. Keep the exact method names given.
4. `assertEquals(EXPECTED, ACTUAL, delta)`: expected first, delta for every double.
5. Always save the call result: `Result r = obj.method(...)`. An unassigned field is `null` -> `NullPointerException` (cost 4 tests in SS26).
6. EasyMock: record -> replay -> verify. Forgetting `verify` = "does not fail on wrong implementation".
7. Black-box: single values (2047, 2048, 2049), never ranges, for boundary tests. Expected result = the output word.
8. Modeling: do NOT add elements that the text does not mention (each extra = penalty). Incomplete model = partial parts not graded.
9. Use `Math.min` for a cap, never `%`.
10. In test code write `80.0` not `80` (integer division).
11. Run tests locally before pushing. Only push when they pass.
12. Do not rename test methods (automatic grading).
13. Grade key: below 40% = 5.0, 40% = 4.7, 45% = 4.3, 50% = 4.0, 55% = 3.7, 60% = 3.3, 65% = 3.0, 70% = 2.7, 75% = 2.3, 80% = 2.0 (bonus does not apply to the retake).

#history WHAT EACH EXAM CONTAINED (so you know what to expect)

Sitting	Parts and points
SS25 Final (Jul 2025)	Quiz 25, Testing (ShoppingCart JUnit) 15, REST 22 (server + WebFlux client + sorting), Mocking (EasyMock Spaceship/Docking) 18, Modeling 20 (activity diagram, thesis workflow)
Retake (Oct 2025)	Quiz 20, Modeling 15 (communication diagram, thesis), Patterns 15 (Observer, ETF charts), Testing 15 (Library JUnit), Mocking 15 (Vehicle EasyMock), REST 20 (pullover stock)
SS26 (Aug 2026)	Unit Testing 20 (CloudServer), Blackbox 15 (EP + BVA), REST 25 (tickets), Quiz 20, Modeling 20 (deployment diagram)
Mock exams (earlier chats)	Acceptance tests (Gherkin) 5, Requirements elicitation 8, Mocking 25 (argument capture), Black box 10, SOLID programming 25, UML use case diagram 7, Quiz 20 (same 20 questions in both mocks)

Repeating themes: JUnit tests, EasyMock, REST (Spring), black-box EP/BVA, one UML diagram (type changes every time), a design pattern or SOLID task, and a quiz. Quiz questions repeat across sittings (INVEST testability, Git, diagrams, coverage, SOLID). Learn #quizbank.

#junit JUNIT 5

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class MyTest {
    private static final double DELTA = 0.0001;
    private MyClass obj;

    @BeforeEach void setUp() { obj = new MyClass(50.0, Tier.A, 2); }

    @Test
    void testSomething() {
        Result r = obj.doIt(Tier.A, 5.0);           // ACT, keep the result
        assertTrue(r.isSuccess());                 // ASSERT
        assertEquals(3.0, r.getCost(), DELTA);
        assertEquals(47.0, obj.getRam(), DELTA);
    }
}
```

Call	Use
assertTrue/False(c)	booleans
assertEquals(exp, act)	int, long, String, enum
assertEquals(exp, act, delta)	double, float
assertNull/NotNull(x)	null checks
assertSame(a, b)	same object
assertThrows(Ex.class, () -> code)	exception expected (EXAM: <code>assertThrows(IllegalArgumentException.class, () -> user.setAge(-10));</code>)
assertAll(...)	several checks, all reported
fail("msg")	force fail
assertTrue(cond, () -> "message")	assertion message (lazy)

Annotations: `@BeforeEach/@AfterEach` (every test), `@BeforeAll/@AfterAll` (static, once), `@Disabled`, `@DisplayName`, `@Timeout`, `@ParameterizedTest + @ValueSource / @CsvSource`. JUnit 5 = Platform + Jupiter + Vintage (Vintage runs old JUnit 3/4 tests). Test method: starts with `test`, calls method under test, checks with `assert`. Class per class or per feature (LECTURE). Four-phase test: Setup, Exercise, Verify, Teardown (LECTURE).

What every method needs

1. normal case for EVERY branch
2. failure case (state must stay unchanged)
3. edge case (exactly at limit, zero, cap). If the task says "verify count AND content", check both. A test that passes on a broken implementation loses points, so assert the real change.

Typical exam test bodies (EXAM)

- Initial state: `assertEquals(0, lib.getBookCount()); assertEquals(1, lib.getMemberCount());` (SS25/Retake lost points for not checking the member count correctly).
- Add multiple: add 2-3 items, check exact count and exact sum.
- Remove: add, remove, check count went down AND the item is gone.
- Null: `assertThrows(IllegalArgumentException.class, () -> cart.addProduct(null));`
- Already borrowed: first borrow works, second borrow `assertThrows(IllegalStateException.class, ...)`.
- Discount: compute expected by hand $(total * (1 - 0.10))$, compare with delta.
- Empty cart (SS25 lost): "empty" may be `getProductCount() == 0` (int), not an empty list: read the UML and the getter type.

Gotcha: integer division

$(2 * 80) / (5 + 10) + 40 = 50$ (WRONG). $(2 * 80.0) / (5.0 + 10.0) + 40.0 = 50.667$ (RIGHT).

SS26 CloudServer (EXAM, full working solution)

```
public AllocationResult allocateTask(ServerTier taskTier, double taskLoad) {
    boolean same = (serverTier == taskTier);
    double actualRamCost = same ? taskLoad * 0.6 : taskLoad * 1.5;
    if (ram < actualRamCost) return new AllocationResult(false, 0.0, 0.0); // ram unchanged
    ram -= actualRamCost;
    double speed = (optimizationLevel * 80.0) / (taskLoad + 10.0);
    if (same) speed += 40.0;
    return new AllocationResult(true, speed, actualRamCost);
}

public double freeRam(double amount) {
    if (amount <= 0.0) return ram;
    double effective = amount * (1.2 + optimizationLevel * 0.05);
    double cap = 600.0 + optimizationLevel * 40.0;
    ram = Math.min(ram + effective, cap);
    return ram;
}
```

Tests (server 50, HIGH_MEM, level 2, task HIGH_MEM load 5): cost 3.0, ram 47.0, speed $160/15+40 = 50.667$. Different tier: cost 7.5, ram 42.5, speed 10.667. Insufficient: ram 0 -> false, ram 0, speed 0, cost 0. freeRam(10) with ram 10, level 2 -> $10 + 13 = 23.0$. Cap: 680.0.

#easymock EASYMOCK

Pattern: Record -> Replay -> Verify

```
@ExtendWith(EasyMockExtension.class)
class VehicleManagementSystemTest {
    @TestSubject private VehicleManagementSystem vehicleManagementSystem = new
VehicleManagementSystem();
    @Mock private Vehicle vehicleMock;    // default mock (NOT nice, not strict)

    @Test
    void testAssignDriverSuccessful() {
        Driver driver = new Driver();
        expect(vehicleMock.assign(driver)).andReturn(true);    // expect = stub AND demand
        replay(vehicleMock);
        vehicleManagementSystem.assign(driver, vehicleMock);
        assertEquals(1, driver.getAssignedVehicles().size()); // check REAL behavior (state)
        verify(vehicleMock);    // proves assign() was really
called
    }
}
```

import static org.easymock.EasyMock.*; Check HOW the SUT gets the mock. Field/setter: @TestSubject injects by type. Constructor injection: build it yourself new Sut(mock) (feedback "you did not configure the test subject correctly").

The 3 exam traps (EXAM, lost points each time)

1. **Failure test** (mock returns false): expect(mock.assign(driver)).andReturn(false); replay; call; verify; AND assert the state (driver has NO vehicle). Without verify or without the state assert it "does not fail on wrong implementation".
2. **Method must NOT be called** (checkEngine false -> assign never called): expect(mock.checkEngine()).andReturn(false); replay(mock); call; verify(mock); Do NOT record assign. On a default (non-nice) mock an unexpected call throws AssertionError, so a wrong implementation fails the test.
3. **Dock/dispatch returns false** (SS25): same as 1, plus verify that dock(astronaut) was executed.

Recording options

Need	Code
return	expect(m.f(x)).andReturn(v);
void method	m.f(x); expectLastCall();
throw	expect(m.f(x)).andThrow(new IllegalStateException());
void throws	m.f(x); expectLastCall().andThrow(new RuntimeException());
count	.once() .times(2) .times(1,3) .atLeastOnce() .anyTimes()
sequence	.andReturn(1).andReturn(2)
any arg	anyInt() anyString() anyObject() isA(X.class) eq(5) gt(3)
capture arg	Capture<Driver> c = new Capture(); expect(m.assign(capture(c))).andReturn(true); ... assertSame(driver, c.getValue());
answer by arg	.andAnswer(() -> ...)

Need	Code
reset	<code>reset(m)</code>

Mock types

Type	Unexpected call	Order
default @Mock / createMock	AssertionError	free
@Mock(type = MockType.STRICT)	AssertionError	must match
@Mock(type = MockType.NICE)	returns 0/null/false	free

Rules: do not mix matchers and plain values (`m.f(anyInt(), eq(5))`). No replay -> "missing behavior definition". No verify -> missing calls undetected. Test doubles (LECTURE): Dummy (filler), Stub (canned answers to SUT), Spy (records what SUT sent out), Mock (checks expected interaction), Fake (light working version, e.g. in-memory DB). Mock needed for behavior/interaction checks; stub or fake is enough for plain state checks.

#whitebox WHITE-BOX AND COVERAGE (EXAM: quiz asks you to compute %)

Coverage types (weak -> strong): statement, branch, term/condition, path. Even 100% coverage does not prove correctness. Statement coverage = every statement runs once. Branch = every if/loop goes both ways. Path = every path (can be infinite with loops).

How to compute "line coverage %" (EXAM)

1. Count the executable lines (lines that do something: `if`, `return`, assignments; not `{`, `}`, method header).
2. Mark which lines the given tests reach.
3. covered / total. Example PasswordValidator (EXAM): executable lines 3,4,6,7,9,10,12 = 7. Tests: valid (3,6,9,12), tooShort (3,6,7), null (3,4). Never reached: line 10 (`return false` for "no digit"). 6/7 = ~85%. Example Calculator (EXAM): add, subtract, multiply, divide, divide by zero are all tested -> 100%. findMin (EXAM): to improve coverage test (a) normal array (min found) and (b) null array (exception). Empty array throws (it does not return 0). NaN and performance tests are irrelevant.

Other facts (LECTURE): test levels unit/module, integration, system, acceptance. Test automation pyramid: many unit tests, fewer integration, few system/acceptance. Test = destructive (successful test = finds a bug). Regression test = rerun to catch re-broken old features. Exploratory test = manual, creative, finds new defects. Tests driven by: requirements (black box), structure (white box), statistics, risk. Test automation does not replace manual testing. Test cannot show absence of errors, cannot show the cause of an error. Load/performance/stress test: performance = single function under base load; load = whole process chain; stress = beyond the limit (does it recover?). Security test = negative tests (penetration test, white-hat hacker). Usability test = real users with tasks (video, eye tracking, logs). Integration styles: Big-Bang (no doubles needed but system rarely runs, failures only at the end), incremental (top-down/bottom-up with doubles), continuous integration (today).

#bva #ep BLACK-BOX: EQUIVALENCE CLASSES AND BOUNDARIES

Method

1. List each input. 2. Split into valid and invalid classes (below, above, wrong type, null, empty). 3. One representative value per class. Cover all classes with the minimum number of test cases: valid classes may share a test, but for INVALID classes only ONE invalid input per test (everything else valid). 4. Boundary values (three-point): for each boundary use n-1, n, n+1 as single numbers. 5. Do not split classes without a reason, redundant test cases lose points (EXAM: "TC1 redundant with TC6"). Table format (LECTURE): Class | Input | Expected output. Expected output is mandatory.

Worked exam: Graphics Configurator (EXAM KEY)

Rules: deviceType "Mobile" or "Console" else unsupported. Mobile 2048..4096 -> performance, 4097..16384 -> quality. Console 2048..8192 -> performance, 8193..16384 -> quality. vramMB <= 2047 or >= 16385 -> unsupported.

Class	Valid/Invalid	Inputs
DC1	Valid	Mobile
DC2	Valid	Console
DC3	Invalid	any other input (Handheld, PC, empty)
VC1	Invalid	vramMB <= 2047
VC2	Invalid	vramMB >= 16385
VC3	Valid	deviceType = Mobile with 4097 <= vramMB <= 16384
VC4	Valid	deviceType = Mobile with 2048 <= vramMB <= 4096
VC5	Valid	deviceType = Console with 2048 <= vramMB <= 8192
VC6	Valid	deviceType = Console with 8193 <= vramMB <= 16384

TC	deviceType	vramMB	Expected
TC1	Mobile	1000	unsupported (VC1)
TC2	Console	3000	performance
TC3	Mobile	6000	quality
TC4	Mobile	3000	performance
TC5	Console	12000	quality
TC6	Console	24000	unsupported (VC2)
TC7	Handheld	6000	unsupported (DC3)

Boundary (Mobile minimum 2048): TC10 = 2047 unsupported, TC11 = 2048 performance, TC12 = 2049 performance. Other boundaries: 4096/4097, 8192/8193, 16384/16385. Do NOT call "Mobile with vramMB > 4096" invalid (it is the valid quality range). Write the exact output word, never inclusive.

LECTURE example: factorial

Classes: negative (-5 -> error), not integer (3.14 -> error), too large (100 -> error), not a number (ABC -> error), normal (7 -> 5040), zero (0 -> 1). Boundary around 12: test 11, 12, 13. ATM withdraw, overdraft up to 2000: boundaries -2000 (ok) and -2001 (reject).

#rest #spring REST SERVER WITH SPRING WEB

Status codes SS26 (EXAM)

POST created 200 OK | POST body already has ID 400 | GET by id found 200 / missing 404 | GET all always 200 | PUT updated 200 / path ID != body ID 400 / missing 404 | DELETE always 204. Follow the task table exactly (success may be 200, not 201). Check mismatch (400) BEFORE not found (404).

Resource (controller) layer

```
@RestController
@RequestMapping("/tickets")
public class TicketResource {
    private final TicketService ticketService;
    public TicketResource(TicketService s) { this.ticketService = s; }

    @PostMapping
    public ResponseEntity<Ticket> createTicket(@RequestBody Ticket ticket) {
        if (ticket.getId() != null) return ResponseEntity.badRequest().build();
        return ResponseEntity.ok(ticketService.saveTicket(ticket));
    }

    @GetMapping("/{ticketId}")
    public ResponseEntity<Ticket> getTicket(@PathVariable Long ticketId) {
        Ticket t = ticketService.findTicketById(ticketId);
        return t == null ? ResponseEntity.notFound().build() : ResponseEntity.ok(t);
    }

    @GetMapping
    public ResponseEntity<List<Ticket>> getAllTickets() { return
ResponseEntity.ok(ticketService.getAllTickets()); }

    @PutMapping("/{ticketId}")
    public ResponseEntity<Ticket> updateTicket(@PathVariable Long ticketId, @RequestBody
Ticket ticket) {
        if (!ticketId.equals(ticket.getId())) return ResponseEntity.badRequest().build();
        Ticket updated = ticketService.saveTicket(ticket);
        return updated == null ? ResponseEntity.notFound().build() :
ResponseEntity.ok(updated);
    }

    @DeleteMapping("/{ticketId}")
    public ResponseEntity<Void> deleteTicket(@PathVariable Long ticketId) {
        ticketService.deleteTicket(ticketId);
        return ResponseEntity.noContent().build();
    }
}
```

Service layer

```
@Service
public class TicketService {
    private final List<Ticket> tickets = new ArrayList<>();
    private long idCounter = 1; // use the counter already in the
project

    public Ticket saveTicket(Ticket t) {
        if (t.getId() == null) { t.setId(idCounter++); tickets.add(t); return t; } // create
        Ticket existing = findTicketById(t.getId()); // update
        if (existing == null) return null; // must NOT silently create
        existing.setTitle(t.getTitle()); // copy ALL fields
        return existing;
    }
    public Ticket findTicketById(Long id) { for (Ticket t : tickets) if (t.getId().equals(id))
return t; return null; }
    public List<Ticket> getAllTickets() { return tickets; }
    public void deleteTicket(Long id) { tickets.removeIf(t -> t.getId().equals(id)); }
}
```

Variants seen in exams (EXAM)

Exceptions instead of ResponseEntity (Retake pullover):

```
@PostMapping("/pullover-stocks")
public PulloverStock createPulloverStock(@RequestBody PulloverStock p) {
    if (p.getId() != null) throw new RuntimeException(HttpStatus.BAD_REQUEST);
    return service.create(p);
}
@PutMapping("/pullover-stocks/{pulloverStockId}/buy") // buy = change state: PUT/POST
on a sub-resource
public PulloverStock buyPullover(@PathVariable Long pulloverStockId) {
    PulloverStock s = service.find(pulloverStockId);
    if (s == null) throw new RuntimeException(HttpStatus.NOT_FOUND);
    if (s.getQuantity() == 0) throw new RuntimeException(HttpStatus.BAD_REQUEST);
    return service.buyPullover(s); // service decreases quantity
}
@GetMapping("/pullover-stocks")
public List<PulloverStock> getAllPulloverStocks(@RequestParam(defaultValue = "false") boolean
onlyAvailable) {
    return service.getAll(onlyAvailable); // repository filters quantity >
0
}
```

Optional sorting params (SS25): `@RequestParam(required = false, defaultValue = "ID")`
`ProductSortingOptions.SortField sortField, @RequestParam(defaultValue = "ASCENDING") SortingOrder`
`sortingOrder`. Service sorts: `list.sort(comparator)`; descending: `comparator.reversed()`. Default = sort
by ID ascending. **Closed layered architecture:** Resource calls ONLY Service, Service calls ONLY
Repository. Never skip a layer. Only change the classes the task names.

#webflux CLIENT with WebClient (SS25 exam, async)

Requirements: a `WebClient`, a local `List<Product>` cache, non-blocking, update the local list after each
success, then call the `Consumer<List<Product>>` with the updated list.

```

private final WebClient webClient = WebClient.create("http://localhost:8080");
private final List<Product> products = new ArrayList<>();

public void addProduct(Product p, Consumer<List<Product>> consumer) {
    webClient.post().uri("/products").bodyValue(p)
        .retrieve().bodyToMono(Product.class)
        .subscribe(created -> { products.add(created); consumer.accept(products); });
}

public void updateProduct(Product p, Consumer<List<Product>> consumer) {
    webClient.put().uri("/products/{id}", p.getId()).bodyValue(p)
        .retrieve().bodyToMono(Product.class)
        .subscribe(upd -> { products.replaceAll(x -> x.getId().equals(upd.getId()) ? upd : x);
consumer.accept(products); });
}

public void deleteProduct(Long id, Consumer<List<Product>> consumer) {
    webClient.delete().uri("/products/{id}", id)
        .retrieve().toBodilessEntity()
        .subscribe(r -> { products.removeIf(x -> x.getId().equals(id));
consumer.accept(products); });
}

public void getAllProducts(Consumer<List<Product>> consumer) {
    webClient.get().uri("/products").retrieve().bodyToFlux(Product.class).collectList()
        .subscribe(list -> { products.clear(); products.addAll(list);
consumer.accept(products); });
}

// sorting: .uri(b -> b.path("/products").queryParam("sortField",
f).queryParam("sortingOrder", o).build())

```

Do not call `.block()` (task says non-blocking). Keep the `subscribe` so the request actually fires.

REST basics (LECTURE/GENERAL)

Resources are nouns, verbs are HTTP methods. GET read (safe, idempotent), POST create (not idempotent), PUT replace (idempotent), PATCH partial, DELETE remove (idempotent). Stateless. JSON. Codes: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 500 Server Error. Context mapping between DDD bounded contexts can use RESTful HTTP. Run `TicketManagerServerApplication` (port 8080). Test: `curl.exe -X POST http://localhost:8080/tickets -H "Content-Type: application/json" -d "{\"title\":\"x\"}"` (PowerShell: `curl.exe`). Then COMMIT AND PUSH.

#acceptance #gherkin ACCEPTANCE TESTS AND BDD (LECTURE)

- Acceptance test = customer view, tied to a user story. System test = whole system in a production-like environment (also non-functional: performance, security).
- ATDD cycle: pick user story -> identify acceptance criteria -> write failing acceptance test -> implement until it passes -> refactor tests -> customer acceptance. FitNesse = wiki tables linked to fixtures.
- BDD uses Gherkin (Cucumber). Written so non-programmers understand.

```

Feature: Refund item
  Scenario: Jeff returns a faulty microwave
    Given Jeff has bought a microwave for $100
    And he has a receipt
    When he returns the microwave
    Then Jeff should be refunded $100

```

Given = context, When = action, Then = result, And/But = continue. One scenario per behavior, include a failure scenario (e.g. wrong password -> error message).

- Three Amigos = domain expert + developer + tester write the scenarios together. Anti-pattern: domain experts or developers write scenarios alone.
- Pattern Common Includes: pull repeated SETUP (e.g. login) into one place and include it. Pattern Parameterized Includes: same test, different parameter values.
- Acceptance test table example (user story back of the card = "Confirmation"): Balance old | Withdrawal | Balance new.
- User story 3 Cs (Jeffries): Card, Conversation, Confirmation.

#solid #design DESIGN PRINCIPLES (EXAM: SS25 quiz, mock SOLID exercise 25 pts)

Principle	One line	Smell	Fix
S Single Responsibility	one reason to change	class prints AND calculates	split classes
O Open-Closed	open for extension, closed for modification	switch/if on type, must edit when adding a case	interface + one class per case (EXAM BonusCalculator -> BonusPolicy)
L Liskov	subclass can replace superclass without surprises	subclass throws <code>UnsupportedOperationException</code> , changes behavior (Bicycle extends Vehicle with <code>startEngine</code>)	do not inherit; split hierarchy/interfaces
I Interface Segregation	clients must not depend on methods they do not use	fat interface (deposit, withdraw, <code>printStatement</code> , <code>requestLoan</code>)	several small interfaces
D Dependency Inversion	high-level depends on abstraction, not on concrete low-level	<code>private MySQLDatabase db = new MySQLDatabase();</code>	Database interface + inject it (EXAM)

Others: Information hiding (private fields, getters/setters: `Point x,y public -> private + getX()/setCartesian()`). Law of Demeter (no `a.getB().getC().doIt()`; talk only to own fields, parameters, own objects). Command-Query Separation (a method either changes state or answers, not both: `username(String) -> setUsername + getUsername`, EXAM). DRY (no clones). Small methods that do one thing. Few arguments (more than 3 -> object; flag arguments are bad). Coupling = dependencies between classes (low is good). Cohesion = how related things inside a class are (high is good). Metrics: CBO (coupling between objects), LCOM (lack of cohesion). Design by Contract: precondition, postcondition, invariant (OCL, JML). CRC cards: Class, Responsibilities, Collaborators (index cards).

SOLID programming exercise template (mock exam, adapt names)

```
public interface EnrollmentRule { boolean canEnroll(Student s, Course c); }
public class PrerequisiteRule implements EnrollmentRule {
    public boolean canEnroll(Student s, Course c) { return
s.getCompletedCourses().containsAll(c.getPrerequisites()); }
}
public class EnrollmentService {
    private final List<EnrollmentRule> rules; // DIP: depend on
abstraction
    public EnrollmentService(List<EnrollmentRule> rules) { this.rules = rules; } // OCP: new
rule = new class, service untouched
    public boolean enroll(Student s, Course c) {
        if (c.getStudents().contains(s)) return false; // duplicate: no change
        for (EnrollmentRule r : rules) if (!r.canEnroll(s, c)) return false; // ALL rules
must pass (AND)
        c.addStudent(s); s.addCourse(c); // change BOTH only after
all checks
        return true;
    }
}
```

Rules: check everything first, mutate last. On failure NEITHER Student nor Course changes.

Code readability questions (EXAM)

Best improvement = better NAMES (p -> printHighValueOrders, m -> ordersByCustomer; calc -> calculateDiscountedPrice, p,d,r -> price, discountRate, discountAmount). Wrong answers: add lots of comments, inline everything, convert loops to while, make it static, synchronized. "Do not comment bad code, rewrite it." Good comments: copyright, informative, intent, warnings, TODO, Javadoc on public API. Names: intention-revealing, consistent (one name per concept), concise. ~33% of source code is identifiers. Developers spend over 50% of their time on code comprehension.

Refactoring (LECTURE)

Restructure code WITHOUT changing external behavior. Rename, Extract Method/Function/Field (counterpart Inline), Pull-Up Method (counterpart Push-Down), Extract Interface. Fixes code smells: too long methods, too deep nesting, bad names. Do it in small steps with tests. Separate refactoring from adding features. Programming guidelines: naming, formatting, special constructs, comments, compiler warnings. Must be justified and checked (manual + automated). Spaghetti code = goto/jumps. Limit variable scope ("Global Variable Considered Harmful").

#patterns #observer DESIGN PATTERNS (EXAM: Retake Patterns 15 pts = 0, learn this)

Gamma et al. (Gang of Four): Creational (Builder, Abstract Factory, Factory Method, Prototype, Singleton), Structural (Adapter, Bridge, Decorator, Facade, Flyweight, Proxy), Behavioral (Chain of Responsibility, Command, Interpreter, Mediator, Memento, Observer, State, Strategy, Visitor, Template Method).

Observer (Retake exam: ETF notifies line chart and bar chart views)

Use when: one object changes and many others must be told and update themselves. UML (use ONLY what the text mentions, extras are penalized):

<pre><<interface>> Observer + update() ^ (realization) LineChartView BarChartView + update() + update() \ / ----- association "views" 1 ---- * ----- (ETF knows Observer interface only)</pre>	<pre>ETF (Subject) - openingPrice, closingPrice, averagePrice - views : List<Observer> + attach(Observer) / detach(Observer) + notifyObservers() // calls update() on every view + getAveragePrice() / getOpeningPrice() / getClosingPrice() + setPrices(...) // changes state then notifies</pre>
---	--

Flow: new price -> ETF.notifyObservers() -> each view.update() -> view asks ETF for the latest prices (pull). Subject depends only on the Observer interface (low coupling). Write: pattern name Observer, 4 sentences why (explanation is not graded). Model must be COMPLETE, otherwise nothing is graded.

```
interface Observer { void update(); }
class ETF { private final List<Observer> views = new ArrayList<>();
void attach(Observer o){views.add(o);} void detach(Observer o){views.remove(o);}
void setPrices(double open,double close,double avg){ /*store*/ notifyObservers(); }
void notifyObservers(){ for(Observer o: views) o.update(); } }
```

MVC (LECTURE): Model (data and logic) notifies registered views on change (Observer), View (representation), Controller (takes input, calls the model).

Other patterns (LECTURE)

- **Singleton**: only one instance, global access. Private static `theInstance`, `getInstance()` creates on first use, protected/private constructor.
- **Facade**: one simple class in front of a complex subsystem (`HomeTheaterFacade.watchMovie()`).
- **State**: object behaves differently per state; each state is a class with the method (Student: Lecture, Exam, LeisureTime, Party).
- **Strategy**: swap algorithm via interface. **Factory**: create objects without `new` in the client.
- **Adapter**: convert one interface to another. **Decorator**: add behavior by wrapping.
- Pattern description parts: Name, Evaluation, Picture, Context, Problem, Forces, Solution, Result (Alexander). Pattern types: design, architecture, analysis, test, process, anti-patterns (recurring BAD solutions), configuration management patterns.
- Empirical (LECTURE): patterns can help understanding; Observer and Singleton correlated with more defects, Factory with fewer; Visitor does not improve comprehension but reduces maintenance effort.

#architecture ARCHITECTURE (L10)

- High-level design = architecture (strategic), detailed design (tactical). Spec says WHAT, design says HOW (no clean separation in practice).
- Architecture = components + relationships + principles. Component = discrete part. Interface = shared boundary / named set of operations. Views: system view, static view, dynamic view (sequence diagram), distribution (deployment).
- Principles: modularization (core), loose coupling, hierarchical structure, separation of concerns (e.g. business logic separate from technical parts), information hiding (Parnas: expose as little as possible).
- Modularity: change in one component has minimal impact on others. Module = operations and data visible only as allowed. Cohesion high inside, coupling low between.
- Conway's Law: system structure copies the organization's communication structure. Inverse Conway Maneuver: design teams to match the architecture you want (microservices: one team per service).
- Architecture conformance analysis: boxes and arrows, tools detect violations (ConQAT).

- **Layers:** each layer only talks to the adjacent layer, services used via upper interface. Three layers: presentation, application (logic), data persistence. Closed layered = no skipping.
 - **Pipes and Filters:** filters transform data, pipes pass it (compiler: lexer, parser, semantic analysis, code generation).
 - **Microservices:** many small independently deployable services, own function/business capability, talk over network (REST, SOAP, RMI). Pros: scale and deploy independently, different tech. Cons: distributed complexity, granularity hard. Monolith = one deployable unit.
 - Sign that a service should be split (EXAM): different teams change unrelated parts of one codebase causing review/deploy delays.
 - **ADR** (architecture decision record): Y-statement "In the context of X, facing Y, we decided for A and neglected B, to achieve Q, accepting downside D." MADR format in Markdown in Git. An architectural decision = design choice for an architecturally significant requirement (tech, library, feature).
-

#ddd DOMAIN-DRIVEN DESIGN (L11)

- Domain = the subject area. Strategic design (rough structure, priorities) vs tactical design (fine detail).
 - **Bounded Context:** self-contained, internally consistent model; one team, own repo, own DB schema. Same word can mean different things per context (Product in Marketing vs Sales). Without it: Big Ball of Mud.
 - **Ubiquitous Language:** one precise language inside a bounded context (developed with domain experts, scenarios).
 - Subdomains: **Core Domain** (main competitive advantage, invest, do not outsource, EXAM), **Supporting** (needed, specific), **Generic** (buy off-the-shelf or outsource).
 - Context Mapping: how contexts relate (Customer-Supplier), translate language; can use RESTful HTTP.
 - **Entity:** has identity. **Value Object:** immutable, no identity, compared by value (Date, Money). **Aggregate:** entities (+ value objects) with ONE aggregate root as the entry point, a transactional consistency boundary (EXAM: Course root containing Lesson and Quiz). Not a Collection, not UML aggregation.
 - **Domain Event:** something that happened, past tense (SprintFinished). Connects bounded contexts. Saved in a transaction, published after.
 - Event Storming (5 steps): 1 domain events (time line), 2 commands (verbs), 3 entities/aggregates, 4 bounded contexts, 5 views (UIs, roles).
 - Technical parts of a bounded context: input adapter, application service (transactions), domain model, output adapter.
-

#requirements REQUIREMENTS (L02)

- Requirement = need + constraints. Requirements analysis hardest and most costly to fix later (Brooks).
- **Functional (FR)** = input/output behavior (WHAT). **Non-functional (NFR)** = quality, performance, constraints. Rule of thumb for exam: if it has a number like users/speed/uptime or a quality word (usable, secure, fast, reliable) -> NFR. If a user can DO something or the system produces something -> FR. EXAM NFRs: "handle 5,000 users without drop in performance", "1,000 orders per minute under 70% CPU", "usable by people with visual impairments" (usability), "stream without buffering on 5 Mbps", "99.9% uptime", "10,000 requests per minute", "user-friendly", "back up all data every 24h". EXAM FRs: monthly billing reports, search by cuisine, save favorites, push notification when picked up, track driver on map, grade quizzes,

enroll in courses, issue certificate. Vague NFR can become FR when concrete: "data protected" (NFR) -> "uses HTTPS" (FR).

- Glinz classification: project, system, process requirements; system = functional, attribute (quality, performance), constraint (fixed, no justification, e.g. use MySQL).
- Process: Collect -> Analyze (classify, prioritize/negotiate, document) -> Specify -> Validate.
- Elicitation techniques: document analysis, observation, surveys (closed/open), interviews (most effective, structured), story-writing workshop, experiments, prototyping, personas. Ask open questions first. Interviews: prepare guideline, two interviewers.
- **Persona**: fictional detailed user (name, background, goals, challenges). Choose design that fits the persona (EXAM: busy developer Luis -> adjustable difficulty that auto-scales on busy days, audio-only mode; NOT permanent streak animations, NOT forced leaderboard).
- Specification methods: free text, structured text, glossary, data/structure/interaction/state/deployment models, use cases, formal. SRS (IEEE 830): 1 Introduction, 2 Overall description, 3 Specific requirements (largest part: interfaces, functional, performance, design constraints, attributes).
- Validation: stakeholder review, inspection with checklist, prototyping, gap analysis, tests.
- **INVEST** (Wake): Independent, Negotiable, Valuable, Estimable, Small (sized appropriately), Testable. TESTABILITY violated = vague, not measurable (EXAM x3: "game should feel exciting", "feed engaging and fun", "error messages helpful"). Good stories: "upload a profile picture", "approve articles", "view past orders", "reset passwords".
- User story: "As a , I want so that ". Epic -> feature -> sprintable story -> task. Progressive refinement (details later).
- Definition of Done: checklist (code reviewed, tested, documented, zero known defects...). NFRs can go into the DoD.
- **Use case**: interaction between actor and system to reach a goal. Fields: Name, Goal, Precondition, Postcondition (also in exceptional case), Actors (main actor), Main success scenario (numbered steps), Exceptional cases (2a, 3a with numbered steps). Use case diagram see #usecase.
- Requirements elicitation exam task (mock): name stakeholders, goals, clarification questions (open), conflicts between stakeholders.

#quality SOFTWARE QUALITY (L05)

- Garvin approaches: transcendental ("know it when I see it"), user/value-based, product-based (measurable), manufacturing-based (process). Crosby: conformance to requirements. Quality depends on the stakeholder (customer, user, developer, operator).
- **ISO 25010 product quality**: Functional suitability (completeness, correctness, appropriateness), Reliability (faultlessness, availability, fault tolerance, recoverability), Performance efficiency (time behaviour, resource utilization, capacity), Interaction capability/Usability (learnability, operability, user error protection, inclusivity...), Maintainability (modularity, reusability, analysability, modifiability, testability), Security (confidentiality, integrity, non-repudiation, accountability, authenticity), Compatibility (co-existence, interoperability), Safety (operational constraint, fail safe, hazard warning...), Flexibility/Portability (adaptability, scalability, installability, replaceability).
- Definitions: **Reliability** = probability of failure-free operation for a time in an environment. **Usability** (ISO 9241-11) = effectiveness, efficiency, satisfaction. **Safety** = cannot reach a state that threatens the environment/people. **Security** = confidentiality, integrity, availability despite attackers. **Performance** = functions within constraints (speed, accuracy, memory). EXAM: confidential data, MFA, resist XSS/CSRF/SQL injection = Security. App works but users cannot find tasks without manual = Usability.
- Safety vs security: safety = can the system hurt someone; security = can someone hurt the system.

- Defect terminology: **Mistake** (human action) -> **Fault/defect/bug** (in code) -> **Error** (wrong internal state) -> **Failure** (user sees wrong behavior). "Defect" = generic.
 - Famous examples: Ariane 5 (reuse without tests, redundancy useless), Therac-25 (safety), Heartbleed (security, shared library), healthcare.gov (performance).
 - Software redundancy is ineffective (different teams make similar mistakes).
 - Avizienis model: dependability = availability, reliability, safety, confidentiality, integrity, maintainability.
-

#qa QUALITY ASSURANCE (L07)

- QA double meaning: "improve" and "show that it is good". Taxonomy: organizational, constructive, analytical. Analytical: non-mechanical (reviews) vs mechanical (static analysis, dynamic test).
 - **Validation** = right system? (user view). **Verification** = system right? (requirements). Memory: validation = "Are we building the right product", verification = "Are we building the product right".
 - False negative (missed fault) is much worse than false positive.
 - Review types by formality: peer review < walkthrough (author explains) < technical review < Fagan inspection (formal).
 - Roles: moderator (leads, not also a reviewer), recorder, reviewers, author, (manager not present). Inspection process: planning, kick-off, individual checking, logging meeting, edit/follow-up, with entry/exit criteria.
 - **Review meeting rules** (EXAM): focus on finding and logging defects, NOT solving them. Discuss the artifact, not the author. Max 2 hours. Findings categories critical/major/minor/good. Recommendation: accept, accept with changes, reject. Optimal reading ~1 page per hour. Style issues beyond guidelines are not discussed.
 - Modern reviews: merge/pull request triggers review (pattern Merge or Pull Requests, Peer-Reviewed Commit).
 - Evidence: reviewers find about 1/3 of faults (up to 93%), 1 to 2 person-hours per fault. Defect pattern tools find about 1/4 of faults, up to 7 person-hours configuration, under 0.5 h per fault, many false positives.
 - **Automated static analysis (ASA)** = examine code without running it (AST, control/data flow, defect patterns, style checker). Tools: Checkstyle, SpotBugs (== on Strings), PMD, SonarQube (dashboard). EXAM: find vulnerabilities, memory leaks, null pointers on every pull request BEFORE build -> ASA.
 - Test: partial, objective, execution. Review: partial, subjective, understanding. ASA: complete, mechanical, abstraction.
 - Pair programming: Driver writes, Navigator reviews (continuous review).
 - LLM for coding: hallucinations, wrong answers (52% in a study), ethics. Prompt: include details, adopt a persona (system prompt).
 - Wagner removal costs: unit 3.5, integration 5.4, system 8.4 staff-hours per defect (later = costlier).
-

#ci #deploy CI, BUILD, DELIVERY (L05, L06)

- Automated build: compile, link, static analysis, tests, deploy, driven by a **build script** (Gradle, Make, Maven). Gradle concepts: project, build script, dependency management, tasks, plugins. `libs.versions.toml` centralizes versions. Tasks: assemble, build, check, test, javadoc, run. JaCoCo plugin = coverage report (`finalizedBy jacocoTestReport`).

- **CI** = integrate often (at least daily) on a CI server, automatically build + test + analysis + notify. EXAM: features work on laptops but break at sprint end -> Continuous Integration. Not CD, not TDD.
- CI pipeline stages (fail fast): build, unit test, integration test, system test, acceptance test, static analysis. Slow performance/stress tests last.
- GitLab CI: `.gitlab-ci.yml`, stages, jobs, runner (Docker image), cache. **CI config** = WHEN and in which environment (knows runtime, calls the build script). **Build script** = WHAT to do (knows repo structure).
- CI reduces risks: no shippable software, works-on-my-machine, late failure detection, no regression testing, low quality, no visibility.
- Big-Bang integration: no doubles needed but failures only at the end.
- **Continuous Delivery** = every change automatically built and tested up to staging, deployment to production is MANUAL. **Continuous Deployment** = automatic deployment to production too. Deployment pipeline: commit stage, acceptance stage (smoke tests, acceptance tests), capacity stage, UAT, production; artifact repository for binaries.
- "If it hurts, do it more often": frequent small releases = small risk.
- Upgrading a critical library (EXAM): run the full automated regression suite in CI, block merge on failure.

Deployment patterns (EXAM: dark launching)

Pattern	Problem	Idea
Blue-Green	quick cut-over and rollback	two production systems (old and new), switch the router
Canary release	faulty version hits all users	small % of users first (e.g. 5%), monitor, then all
Dark launching	test new backend on real traffic with ZERO user impact	new component gets silent copies of real requests, result unused, user sees old version
A/B testing	which variant is better	users split between variant A and B, compare metrics
Feature toggle	unfinished features on mainline	switch features on/off by config (feature flags)

Feature branches vs feature toggles + mainline: long branches hurt CI; toggles keep mainline integrated.

#git GIT (L03, EXAM)

- Three states: **modified** (changed, not staged), **staged** (marked for next commit), **committed** (saved in local DB). `git add` -> Staged (EXAM). Untracked = new file git does not track.
- Snapshots, SHA-1 checksums (40 hex), almost all operations local. Branch = pointer to a commit, HEAD = current branch.
- Commands: `git status`, `git add <f>`, `git commit -m ""`, `git commit -a -m`, `git branch x`, `git checkout x`, `git checkout -b x`, `git merge x`, `git tag -a v1.4 -m ""`, `git log`, `git diff`, `git stash`, `git clone`, `git fetch`, `git pull`, `git push`.
- **fetch** = download remote branches/commits, changes NOTHING local (EXAM: Eleanor wants to see the new button branch safely -> `git fetch`). **pull** = fetch + merge. **push** = upload your commits.
- New local branch, no tracking info (EXAM): `git push` (really `git push -u origin feature/navbar`), NOT `git pull`.
- Push rejected non-fast-forward (EXAM): remote has work you do not have -> `git pull` (integrate), then push. NOT `--force`, NOT `reset --hard`, NOT a new branch.

- Forgot a file in the last commit, not pushed (EXAM answer): `git add AuthREADME.md -> git commit -m "..."` (a new commit). (Also valid in practice: `git commit --amend.`)
 - Merge: fast-forward (no divergence) vs merge commit (recursive). Conflicts resolved manually. Tags: lightweight (pointer) vs annotated (object with author/date/message), annotated tag = a configuration.
 - Commit message: first line 50 chars max, blank line, body wrapped at 72.
 - Configuration management = identify and manage versions/variants/configurations, reproduce old ones. Maven directory layout: `src/main/java`, `src/test/java`, `target/`.
 - **Branching patterns** (Fowler): Source Branching, Mainline, Healthy Branch (always validated by automated checks), Mainline Integration, Feature Branch, Continuous Integration (integrate at least daily), Peer-Reviewed Commit, Release Branch (only stabilization), Maturity Branch.
-

#process PROCESS MODELS, SCRUM, MANAGEMENT (L08, L09)

Life cycle models

- **Waterfall**: sequential phases (analysis, spec, architecture, detailed design/coding, integration/test, acceptance, operation), little iteration. EXAM: fixed requirements, formal sign-off per phase, no changes -> Waterfall.
- **Prototyping**: mock-ups to get early feedback on requirements. Floyd: exploratory (supports analysis), experimental (technical), evolutionary (really a way of developing).
- **Iterative**: repeat analysis-design-code-test, improve the existing system each round (we get things wrong before we get them right).
- **Incremental**: build a core system first, each increment adds functionality, delivered in usable parts (NOT delivered all at the end). ISO: overlapping, iterative rather than sequential.
- **Spiral** (Boehm): risk driven, a generic approach.
- V-model: test phases mirror dev phases (GENERAL).
- Project -> product shift: Project leader, Product Owner, Scrum Master (process). DevOps joins Dev and Ops.
- **Maintenance types** (EXAM: dark mode = perfective): corrective (fix bugs), adaptive (new environment/technology: new DB, OS), perfective (new features or improvements), preventive (improve structure, refactor). Software maintenance mainly deals with changes in requirements and technology.

Scrum (LECTURE)

- Roles: **Product Owner** (product goal, backlog priority, accepts/rejects work), **Scrum Master** (process, removes impediments, shields team, facilitates), **Developers** (5 to 9, cross-functional, self-organizing).
- Events: Sprint (2 to 4 weeks, max 1 month, fixed length), Sprint Planning (max 8 h; why valuable, what, how), Daily Scrum (15 min, 3 questions: yesterday, today, impediments), Sprint Review (max 4 h, demo to stakeholders, no slides), Sprint Retrospective (improve process, max 3 h, start/stop/continue).
- Artifacts: Product Backlog (prioritized by PO), Sprint Backlog (team picks tasks, never assigned), Increment (meets the Definition of Done).
- No changes to the sprint scope during a sprint.
- Critical security bug reported mid-sprint (EXAM): pause, assess impact WITH the Product Owner, then plan next steps (not drop everything, not wait until the end, not escalate to management).
- Agile manifesto: individuals and interactions over processes and tools; working software over comprehensive documentation; customer collaboration over contract negotiation; responding to change over following a plan.

- Scrum of Scrums, SAFe = scaling.
- **Kanban** (GENERAL): board, cards move left to right, WIP limit per stage, continuous flow, NO fixed sprints (sprints = Scrum).
- XP (GENERAL): pair programming, TDD, CI, small releases, refactoring.

Roles

Customer pays, user interacts, stakeholder = anyone with legitimate interest. Analyst/requirements engineer, programmer, architect, tester, DevOps engineer (builds CI/CD), project leader (interface to management), group leader (line supervisor), Product Owner/Manager (represents customer and user).

Management tasks

- Product planning: vision, backlog, roadmap. **Make or buy**: buy/reuse if software does not contain core know-how (core domain: build).
- Kick-off (lift-off): Purpose, Alignment, Context.
- **Risk management**: identify, analyze/evaluate (risk value = probability x cost), plan countermeasures, monitor. Risk register. Boehm top 10 (personnel, unrealistic schedules, wrong functionality, wrong UI, gold plating, requirement changes...).
- Planning: work packages (WBS, tree, doable in about a month), milestones (deliverables + criteria + authority + date), schedule, effort.
- **Effort estimation**: expert vs algorithmic, top-down vs bottom-up, activities vs products. Wideband Delphi (anonymous estimates, discuss differences, repeat). **Planning Poker** (Fibonacci cards, relative, agile). PM = person-month.
- Prospective chart: events by impact and probability.
- Software engineering basics (T01): empirical science, programming in the large, software is immaterial, no natural locality, copy = original, autonomous. Project-based vs product-based. Stand-alone, hybrid, SaaS. Effectiveness, efficiency, productivity. Software crisis (1960s), Royce, Parnas, Boehm, Hamilton coined "software engineering". Shu-Ha-Ri. Defensive discipline. Team vs group. Tuckman stages: forming, storming, norming, performing.
- Software characteristics (EXAM quiz): no natural locality is TRUE; engineers must create distance and isolation (memory does not provide it); reuse is lucrative with little customization, NOT always cheap.

#comprehension CODE COMPREHENSION AND METRICS (L06)

- Mental model, cognitive model; chunking, beacons, plans, hypotheses; top-down and bottom-up (von Mayrhauser and Vans). Inherent complexity (domain) vs incidental complexity (bad design).
- **Cyclomatic complexity** (McCabe) $V(g) = E - N + 2P$ (edges, nodes, connected components), or decision points + 1. Measures independent paths = testability. Low or no correlation with understandability (Scalabrino).
- **Cognitive complexity** (SonarSource): +1 for each break in linear flow (if, else if, else, switch, loops, catch, goto, sequences of logical operators, recursion) AND extra +nesting level for nested ones. **NO increment for a method declaration**, no increment for shorthand structures. Better correlation with understanding time ($r = 0.54$). EXAM: cognitive complexity increases when linear flow is interrupted and when nesting increases. Example: for + nested if = $1 + (1+1 \text{ nesting}) = 3$ cognitive; cyclomatic = 3 (2 decisions + 1).
- Details: detail complexity (number of parts) vs dynamic complexity (unclear cause and effect). Abstraction goals: reduce complexity, increase reuse. Generalizing abstraction (type, add a

parameter) vs simplifying abstraction (word, remove parameter).

#modelprops #uml MODELS AND UML (L09)

Model properties (Stachowiak)

- **Mapping**: model represents an original (a "Patient" class stands for real patients).
- **Reduction / abstraction**: model leaves out details (map without elevation lines).
- **Pragmatics**: model has a purpose for certain users/time (highways only for long trips).
- **Descriptive model**: describes something that EXISTS (photo, sequence diagram of how an order works now). **Prescriptive model**: describes something TO BE MADE (blueprint, "only managers can approve leave"). The SS26 matching question (5 scenarios) gave only 0.2/1 and the key was not shown, so decide scenario by scenario with the definitions above.

Diagram overview

Structure: class, package, object, composite structure, component, deployment. Behavior: use case, activity, state, sequence, communication, timing, interaction overview. Pick the diagram (EXAM): order of service calls and responses between services = **sequence diagram** (x2). Workflow with decision points from start to end = **activity diagram**. Larger parts and interfaces = component. Hardware nodes = deployment. States of an object = state machine. Application functionality from the user view = use case. Communication diagram = like sequence but objects and links, good for many objects and little message flow. Tools: PlantUML (text-based, versionable).

#deployment Deployment diagram (SS26 EXAM, 4/20)

- Node (3D box): `<<device>>` = physical hardware (Smart Badge, Dispensing Cart, Transport Terminal). A server computer = **machine** node (EXAM: Central Pharmacy Server was wrongly a device). Others: `<<executionEnvironment>>`.
- **Artifact** = file/deployable (`<<artifact>>`, Cart UI, Staff Identity Token). It is NOT a component. Put artifacts INSIDE their node.
- **Component** = software part with interfaces (Dispenser, Pharmacy Manager), also inside its node.
- Communication path = **deployment association**, a plain line between nodes labeled with the protocol (NFC, TLS, gRPC, HTTPS, JDBC). NO lollipop on it (a protocol is not an interface).
- Interfaces: provided = lollipop (circle), required = socket (half circle). Connect the required socket of one component to the provided lollipop of the other, even across nodes. Name interfaces with service names (e.g. Prescription Authorization Service).
- Do not add or remove elements the text does not mention.
- SS26 system: Smart Badge (artifact Staff Identity Token) -NFC- Dispensing Cart and Transport Terminal. Dispensing Cart (artifact Cart UI, component Dispenser) -TLS- Central Pharmacy Server (machine, component Pharmacy Manager). Transport Terminal (component Dispatch Controller) -gRPC- Central Logistics Server (machine, component Dispatch Manager). Dispenser requires 1 interface from Pharmacy Manager. Dispatch Controller requires 2 interfaces from Dispatch Manager (transport scheduling service, orderly availability update service).

#communication Communication diagram (Retake EXAM, 2.5/15)

- Rectangles = objects written `name:Type` (e.g. `andrei:Student`) or actors; solid lines = links between them; arrows with messages next to the links.
- EVERY MESSAGE MUST BE NUMBERED in order (1, 2, 3, nested 2.1, 2.2). Missing numbers = 0 points ("not a functional diagram").

- Write the explanation (scenario name, list of actors, list of the message flow) if the task asks.
- Only use objects/messages from the text, each extra is penalized. Do not add attributes unless asked.
- Thesis scenario draft (adapt to the exact text): 1 selectTopic(LLMs) andrei -> topic; 2 registerThesis() stefan -> topic; 3 notifyASA() stefan -> denise; 4 [requirements met] confirm() denise -> andrei; 5 confirm() denise -> stefan; 6 writeThesis() andrei -> thesis; 7 submitThesis() andrei -> stefan; 8 reviseThesis() stefan -> thesis; 9 [not correct] provideFeedback() stefan -> andrei; 10 requestExtension() andrei -> denise; 11 updateDeadline() denise -> thesis.
- Sequence diagram: lifelines, activation bars, sync call = solid line filled arrowhead, return = dashed line, async = open arrowhead, create/destroy (X), fragments alt, opt, loop, par, ref.

#activity Activity diagram (SS25 EXAM, 14.5/20, lost 3 + 1 + 0.5 + 1)

- Elements: initial node (filled circle), actions (rounded boxes), decision (diamond, one in, several out with guards [yes]/[no]), merge (diamond, several in, one out), fork (bar, splits into PARALLEL flows), join (bar, waits for all), activity final (circle with dot, ends everything), flow final (X, ends one flow), swimlanes (who does it), object nodes (data/things that flow, NOT sentences like "student cannot do thesis").
- Mistakes from the exam: (1) a merge node where an ACTION followed by a DECISION belonged ("ASA verifies" is an action, then a decision "requirements satisfied?"). (2) fork used for steps that are not parallel (only use when the text says "simultaneously"). (3) a sentence used as an object node. (4) redundant end nodes (one final node is enough unless the text needs more).
- Pattern: action -> decision diamond with guards on both outgoing arrows -> branches. Use a merge only to JOIN branches again.
- Thesis workflow (EXAM): search topic, submit request -> [expectations met?] yes: topic registered, no: continue searching at another chair -> ASA verifies requirements -> [satisfied?] yes: send confirmation emails to student and chair, no: student cannot do thesis (end) -> FORK: do research and write thesis (simultaneous) -> JOIN -> upload thesis -> [correct?] yes: schedule presentation, no: end -> student presents, examiner enters grade -> ASA validates grade -> end.
- Sign-up workflow (EXAM Retake): click Sign Up -> fill form -> [username taken?] yes: show error message, no: email verification -> end.

#class Class diagram (LECTURE)

Box: name / attributes / operations. Visibility + public - private # protected ~ package. Association line with multiplicity (1, 0..1, *, 1..*) and name. Aggregation = hollow diamond, composition = filled diamond (part dies with whole). Inheritance = solid line + hollow triangle. Realization (implements) = dashed line + hollow triangle. Dependency = dashed open arrow. <<interface>>. Object diagram = instances name : Type with values. Webshop domain example: Customer, Order, Book, Copy, Address, PaymentType (CreditCard, BankDetails).

#usecase Use case diagram (mock exam, LECTURE)

Actors (stick figures) OUTSIDE the system boundary box; use cases = ellipses inside. Association actor-use case. <<include>> = base ALWAYS uses it (dashed arrow from base to included, e.g. Withdraw money includes Authenticate). <<extend>> = OPTIONAL extension (dashed arrow from extension to base, e.g. Print statement extends Check balance). Actor can be a person or an external system (Central Bank System). Use cases can be grouped (packages: Information Services, Transactions, Basic Services). Pros: shows who does what, clear for small models. Cons: relations between use cases only rudimentary, no content of the use case (write the table spec for that).

#state State machine

States, initial pseudostate, final state, transition `event [guard] / action`, inside a state `entry/`, `do/`, `exit/`.

Component and composite

Component diagram: components + provided/required interfaces (good for architecture). Composite structure: inside of a component. Package diagram: groups and dependencies.

#quizbank EVERY PAST QUIZ QUESTION WITH ANSWER

SS26 (EXAM KEY)

Topic	Answer
Software characteristics	TRUE: no natural locality; SE must create distance and isolation; reuse lucrative with little customization. FALSE: memory gives protective distance; reuse always cheap
FR/NFR	FR: grade quizzes, enroll, issue certificate. NFR: usable by visually impaired, stream on 5 Mbps, backup every 24h
INVEST "error messages helpful"	Testable
<code>git add README.md</code>	Staged
Test 0 C = 32 F	<code>assertEquals(32.0, result, 0.001)</code> and <code>assertTrue(result == 32.0)</code> (NOT <code>assertEquals(0.0, result)</code>)
Repeated login in acceptance tests	Common Includes
Users cannot figure out basic tasks	Usability
New payment backend on real traffic, no user sees results	Dark Launching
Cognitive complexity increases	linear flow interrupted; nesting increases (NOT method declared, NOT long names)
Find vulnerabilities, null pointers in CI before build	Automated Static Analysis
Review meeting rule	focus on logging defects, solutions later
Process definitions	incremental = usable parts; waterfall = sequential phases; Kanban = board + WIP. Sprints are Scrum; facilitating is Scrum Master
Model properties (5 scenarios)	see #modelprops (key not shown)
Architecture principles	information hiding = private behind interface; high cohesion = related tasks together; modularization = independent reusable modules; low coupling = minimal dependencies; separation of concerns = each part one responsibility
<code>UserService</code> with new <code>MySQLDatabase()</code>	violates Dependency Inversion; fix: <code>Database</code> interface, inject it
Core domain	the main competitive advantage
Maintenance deals with	requirements and technology

Topic	Answer
Add export formats without editing PDF exporter	exporter interface + separate implementation per format

SS25 Final (EXAM KEY)

#	Topic	Answer
1	Persona Luis	auto-scaling difficulty; audio-only mode
2	Functional requirement	monthly billing reports (others are NFR: user-friendly, 10,000 req/min, 99.9% uptime)
3	INVEST testability violated	"gamer wants the game to feel exciting"
4	New local branch, git pull fails	<code>git push</code>
5	See new remote branch safely	<code>git fetch</code>
6	Calculator coverage	100%
7	abs test	<code>assertEquals(3, util.abs(-3))</code>
8	Confidential, MFA, XSS	Security
9	<code>isAllLowercase [a-z]+</code>	only "helloworld" is true (" and digits are false)
10	Method updates AND returns name	split into setUsername/getUsername
11	Readability of <code>p(m)</code>	rename to <code>printHighValueOrders, ordersByCustomer</code>
12	<code>removeDuplicates</code> tests	expected list with duplicates; keeps order of first occurrences
13	Stable search before webinar	unit tests in CI on every commit
14	Forgot file in last commit	<code>git add X -> git commit -m</code>
15	Compile errors	<code>print()</code> signature mismatch; <code>text</code> undefined
16	Critical security bug mid-sprint	pause, assess with Product Owner, plan
17	5 microservices call order	Sequence diagram
18	Fixed requirements, sign-off per phase	Waterfall
19	When to split a service	teams change unrelated parts, delays
20	Which service calls which, in what order	Sequence diagram
21	Public x,y	private + <code>getX()/setCartesian()</code>
22	switch in BonusCalculator	Open-Closed, <code>BonusPolicy</code> interface per role
23	Aggregate Course/Lesson/Quiz	Course = aggregate root containing Lesson and Quiz
24	Dark mode	perfective maintenance

Retake Oct 2025 (EXAM KEY)

#	Topic	Answer
1	INVEST testability	"content feed engaging and fun"
2	push rejected non-fast-forward	git pull, merge, push (not --force)
3	exception test	assertThrows(IllegalArgumentException.class, () -> user.setAge(-10));
4	breaks at end of sprint	Continuous Integration
5	readability of calc(p,d)	rename parameters AND method (no extra comments)
6	upgrade a major library	full automated regression suite in CI, block merge on failure
7	PasswordValidator coverage	~85%
8	Sign-up workflow	Activity diagram
9	[a-zA-Z0-9]{4,10}	"testuser1" and "testUSER4" pass ("abc", too long fail)
10	findMin coverage	normal array + null array handling
11	Compile errors	instance method from static main; String passed to int parameter
12	NFRs	5,000 users without drop; 1,000 orders/min under 70% CPU

#confusion SWAPPED CONCEPTS (if stuck between two options)

A	B
validation = right system	verification = system right
fetch = download only	pull = fetch + merge
staged = marked for commit	modified = changed only
CI = integrate and test often	CD = ready to deploy (manual prod); continuous deployment = automatic prod
canary = few users first	dark launching = hidden copy of traffic, zero user impact
blue-green = two prod systems, switch	feature toggle = switch a feature in config
iterative = repeat and improve	incremental = add parts
Scrum Master = process, impediments	Product Owner = backlog, priorities, accepts work
Scrum = sprints	Kanban = WIP limits, no sprints
FR = what it does	NFR = how well, numbers
fault = bug in code	failure = user sees wrong result; error = bad internal state; mistake = human action
safety = system endangers people	security = attackers endanger system
reliability = failure-free probability	availability = system up when needed
coupling (low) = few dependencies	cohesion (high) = related stuff together

A	B
SRP = one reason to change	ISP = small interfaces; DIP = depend on abstractions; OCP = extend without modify
aggregation (hollow diamond)	composition (filled diamond)
include = always	extend = optional
decision (1 in, many out)	merge (many in, 1 out)
fork = parallel split	join = wait for all
sequence = ordered calls	communication = objects + numbered messages; activity = workflow
component = software part	artifact = file; device = hardware; machine = server node
descriptive = existing	prescriptive = to be built
unit test = one class	integration = interaction
black box = from spec	white box = from code
test stub = feeds input to SUT	spy = records output of SUT; mock = verifies interaction; fake = light real implementation; dummy = filler
Common Includes = shared setup	Parameterized Includes = same test, different data
corrective = fix	adaptive = environment; perfective = improve/features; preventive = restructure
strategic design = bounded contexts	tactical design = aggregates, entities
entity = identity	value object = no identity
cyclomatic = paths (testability)	cognitive = understandability, counts nesting
walkthrough = author explains	inspection = formal roles and process
architecture = what components	detailed design = inside components

#heuristics GUESSING RULES FOR MULTIPLE CHOICE

1. "Always", "never", "impossible", "only", "regardless" in an option: usually wrong.
2. Options that add a new tool, big rewrite, or manual process when a standard practice fits: wrong.
3. "Best explains / most effective": the answer is usually the named principle from the lecture (information hiding, fail fast, one reason to change).
4. Code questions: run the code in your head on each option.
5. Several answers can be right ("choose all"): partial score exists, wrong boxes cost points, so only tick what you are sure of.
6. Metric correlates with understanding? Lecture says naive assumption is false (cyclomatic vs understandability).
7. Vague adjective in a requirement (fun, engaging, helpful, user-friendly): it is untestable and non-functional.
8. A question about "what to do first" in Scrum: involve the Product Owner.
9. "Safest" git answer: never force push, never reset --hard.
10. Which diagram: order of calls = sequence, flow with decisions = activity, hardware = deployment, parts + interfaces = component.

#mistakes POINTS LOST IN REAL EXAMS

Mistake	Fix
REST: no commit (0 in all sittings, 22 + 20 + 25 points)	commit AND push before time ends
Test result variable never assigned (null)	<code>X r = obj.method()</code>
<code>assertEquals</code> args swapped, no delta for doubles	expected first, <code>0.0001</code>
Test checks the wrong thing (member count, empty list)	read UML getters, assert exact counts
Test passes on broken implementation	assert real state change, use verify
EasyMock no verify / not-called case recorded	do not record, replay, verify
EasyMock test subject not configured	<code>@TestSubject</code> created, or construct with the mock
freeRam used <code>%</code> instead of cap	<code>Math.min(value, cap)</code>
Integer division in expected value	use <code>.0</code> literals
Black box: ranges as boundary values	single numbers n-1, n, n+1
Black box: <code>inclusive</code> as expected result	output word
Black box: redundant TC / valid range called invalid	one new class per TC; re-read rules
Quiz: usable/user-friendly as functional	quality = NFR
Quiz: vague story = Estimable	vague = Testable
Observer pattern not recognized (0/15)	"notifies all views, they ask the subject" = Observer
Communication diagram without numbers (0)	number every message
Activity: merge instead of action + decision, fork not parallel	see #activity
Deployment: artifact as component, server as device, lollipop on protocol	see #deployment
Extra elements in a model	only what the text says

Appeals (if needed)

Labeling and wording ambiguities are stronger appeal grounds than "redundant" calls. Grading is per test method, so one root cause can cost several tests: quote the grading rule politely and argue the logic was correct.